

EC143 Final Project: Quantile-Regression Test of Volume Contraction in Qullamaggie Breakouts

Author: Evan Maus

Course: Ec143, Spring 2026

Due: May 14, 2026

I test whether a specific feature of momentum breakouts predicts the upper tail of post-breakout returns more strongly than the median. The feature is the volume contraction ratio across the consolidation base of a Qullamaggie-style setup. The strategy is a quantile regression test statistic, $\beta(0.90) - \beta(0.50)$, with inference from a stationary block bootstrap, on a panel of 5,286 setups in U.S. equities from 2010 to 2025. The hypothesis I locked in before running the test, on the 2010 to 2017 training window, fails. When I ran the test once on my actual data, I got a point estimate of negative 0.0045. The 95% bootstrap confidence interval is [negative 0.038, positive 0.043], which includes zero. A permutation placebo, where I shuffled the volume contraction values to destroy any real link to returns, places my actual estimate at the 39th percentile of that null distribution. So the data is essentially indistinguishable from random.

But the predicted negative sign shows up in 7 of 8 expanding-window out-of-sample years, and the magnitude grows from negative 0.0045 to negative 0.021 as the training window extends to 2024. This is consistent with a small true effect that the 925-row training sample lacked the power to detect, not with no effect at all.

A separate quantile regression sweep on average daily range shows a clean sign flip across tau that OLS hides completely. This illustrates why quantile regression matters, even when the headline pre-registered test fails.

1 Introduction

This project tests whether a specific feature of momentum breakout setups, the volume contraction ratio across the consolidation base, predicts the upper tail of post-breakout returns more than the median. The setting is United States equities from 2010 to 2025. A Qullamaggie-style breakout setup is a stock that has had a strong recent advance, then trades sideways in a tight range (the consolidation base) for several weeks on declining volume, before breaking out above the range high. The test is a quantile regression of factor-residualized 20-day forward returns on volume contraction and three control variables, with inference via stationary block bootstrap. The motivation is that an effect concentrated in the upper tail is invisible to OLS but actionable in a real trading system that

lets winners run and cuts losers, so the conditional-mean and conditional-quantile answers can disagree in a way that matters.

2 Data

The primary panel is daily OHLCV bars (open, high, low, close, volume for each stock on each trading day) for United States equities, sourced from Massive (massive.com), a paid historical equities data provider used in production for `breakouts.trade` and not redistributed in this repository. The sample period is January 2010 to December 2025, with bars from January 2008 onward also loaded so trailing-window features (such as relative strength and consolidation statistics) have enough history to compute on early-2010 setups.

Three additional data sources are pulled to support the empirical specification. First, daily Fama-French three-factor returns (Mkt-RF, SMB, HML) plus the UMD momentum factor are downloaded from Ken French's public data library and merged onto the panel by date; these are used in the residualization stage to strip out market, size, value, and momentum exposure from the forward returns. Second, daily SPY closing prices (split- and dividend-adjusted) are fetched once via `yfinance` and cached locally; SPY is used as the market benchmark for computing relative-strength features. Third, ticker-level metadata (security type and country of origin) is pulled from `yfinance`'s `.info` field and used by the universe filter to keep only US-domiciled common stocks, dropping ADRs, ETFs, and foreign listings. After universe filtering, security-type filtering, and the six-stage breakout detector, the analysis panel contains 5,286 setup-row observations across 1,279 unique tickers, split into 2,214 strict and 3,075 loose universe variants.

```
In [15]: """Notebook setup. Run this cell first; all other code cells assume `REPO` / `DATA`
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import polars as pl
from IPython.display import display

# Resolve the repo root from wherever the notebook was launched.
REPO = Path.cwd()
if not (REPO / "data").exists():
    if (REPO.parent / "data").exists():
        REPO = REPO.parent
    else:
        raise FileNotFoundError(f"Cannot find data/m3_setups_with_residuals.parquet")

DATA = REPO / "data"
FACTORS = REPO / "data"

pl.Config.set_tbl_rows(40)
pl.Config.set_tbl_width_chars(200)
pd.set_option("display.precision", 4)
```

```
print(f"REPO = {REPO}")
print(f"DATA = {DATA}")
```

```
REPO = C:\Users\evanm\Documents\courses\econ143\ec143-project
DATA = C:\Users\evanm\Documents\courses\econ143\ec143-project\data
```

```
In [16]: setups = pl.read_parquet(DATA / "m3_setups_with_residuals.parquet")
print(f"Final analysis panel: {setups.height:,} setup-rows")
print(f"Unique tickers:      {setups['ticker'].n_unique():,}")
print(f"Date range:         {setups['date'].min()} → {setups['date'].max()}")
print()
setups.group_by("universe_variant").agg(pl.len().alias("n")).sort("universe_variant")
```

```
Final analysis panel: 5,286 setup-rows
Unique tickers:      1,279
Date range:         2010-03-24 → 2025-12-24
```

```
Out[16]: shape: (2, 2)
```

universe_variant	n
str	u32
"loose"	3073
"strict"	2213

3 Methods

"The pipeline is implemented as seven stages, each in its own module under `src/`, with each stage saving its output to a parquet file that the next stage reads. The chain runs from raw daily bars (M0 audit), through breakout detection (M1), per-setup feature computation (M2), factor-panel construction and forward-return residualization (M3), training-window OLS and quantile regression (M4), expanding-window walk-forward out-of-sample evaluation (M5), and stationary block bootstrap inference plus a permutation placebo (M6). Key design choices, including the M1 universe filter parameters, the half-split rule for volume contraction, and the bootstrap block length, were locked before any quantile regression coefficients were fit. All random seeds are fixed, so the four parquets in `data/` regenerate every figure and table in this notebook.

Stage	Module	What it does
M0	<code>src/m0_audit.py</code>	survivorship + bad-bar audit on the daily-bar source
M1	<code>src/m1_universe.py</code>	universe filter + 6-stage Qullamaggie consolidation-breakout detector
M2	<code>src/m2_features.py</code>	per-setup features: <code>vol_contraction_ratio</code> , <code>adr_pct</code> , <code>base_duration_days</code> , <code>rs_slope_vs_spy</code> , <code>sector</code>
M3	<code>src/m3_factors.py</code>	FF3+UMD daily factor panel + 20-day-forward residualization on the 2010–2017 training window

Stage	Module	What it does
M4	<code>src/m4_estimation.py</code>	OLS + QR(τ in {0.10, 0.25, 0.50, 0.75, 0.90}) on training
M5	<code>src/m5_walkforward.py</code>	expanding-window OOS sign-consistency, 2018–2025
M6	<code>src/m6_bootstrap.py</code>	stationary block bootstrap CI + placebo (mean block length 30, B=1000)

4 Descriptive statistics

The 5,286 setup observations are not evenly distributed across years. Activity peaks during 2020 and 2021 (the COVID-recovery bull market) and 2024 to 2025, with lighter periods in 2016 and 2022 consistent with the broader market regime. The four candidate covariates have well-behaved distributions inside their expected Qullamaggie ranges: volume contraction ratio has a median of 0.94 and an interquartile range of [0.77, 1.18], average daily range has a median of 3.8% and an IQR of [3.1%, 4.8%], and consolidation duration has a median of 20 trading days and an IQR of [16, 27]. Sector composition is dominated by Healthcare (1,190 setups), Technology (1,072), Consumer Cyclical (669), and Industrials (639), reflecting the cross-section of stocks that produce the underlying breakout pattern.

```
In [17]: # Setup counts by calendar year and universe variant.
(
    setups.with_columns(pl.col("date").dt.year().alias("year"))
        .group_by(["year", "universe_variant"]).agg(pl.len().alias("n"))
        .pivot(values="n", index="year", on="universe_variant")
        .fill_null(0)
        .sort("year")
)
```

Out[17]: shape: (16, 3)

year	loose	strict
i32	u32	u32
2010	116	89
2011	122	94
2012	96	70
2013	138	112
2014	101	87
2015	94	74
2016	84	64
2017	174	136
2018	149	124
2019	155	130
2020	327	204
2021	414	237
2022	140	106
2023	220	166
2024	362	246
2025	381	274

```
In [18]: # Distribution summary of the four candidate covariates by universe variant.
RHS = ["vol_contraction_ratio_w", "adr_pct", "base_duration_days", "rs_slope_vs_spy"]
rows = []
for v in ("loose", "strict"):
    s = setups.filter(pl.col("universe_variant") == v).select(RHS)
    for col in RHS:
        x = s[col]
        rows.append({
            "variant": v, "feature": col,
            "mean": float(x.mean()),
            "std": float(x.std()),
            "p10": float(x.quantile(0.10)),
            "median": float(x.median()),
            "p90": float(x.quantile(0.90)),
        })
pl.DataFrame(rows)
```

Out[18]: shape: (8, 7)

variant	feature	mean	std	p10	median	p90
str	str	f64	f64	f64	f64	f64
"loose"	"vol_contraction_ratio_w"	1.015272	0.375834	0.633849	0.944664	1.471525
"loose"	"adr_pct"	0.041491	0.014688	0.027656	0.037422	0.060033
"loose"	"base_duration_days"	22.555809	7.480893	15.0	20.0	35.0
"loose"	"rs_slope_vs_spy"	0.000748	0.004146	-0.003738	0.000384	0.005784
"strict"	"vol_contraction_ratio_w"	1.022762	0.387345	0.633849	0.942481	1.502001
"strict"	"adr_pct"	0.04281	0.015021	0.028209	0.038736	0.062218
"strict"	"base_duration_days"	22.198373	7.284763	15.0	20.0	34.0
"strict"	"rs_slope_vs_spy"	0.000842	0.004274	-0.003822	0.00043	0.006131

```
In [19]: # Sector composition: counts by variant.
(
    setups.group_by(["sector", "universe_variant"]).agg(pl.len().alias("n"))
    .pivot(values="n", index="sector", on="universe_variant")
    .fill_null(0)
    .with_columns((pl.col("loose") + pl.col("strict")).alias("total"))
    .sort("total", descending=True)
)
```

Out[19]: shape: (12, 4)

sector	strict	loose	total
	str	u32	u32
	u32	u32	u32
"Healthcare"	513	677	1190
"Technology"	445	627	1072
"Consumer Cyclical"	265	404	669
"Industrials"	267	372	639
"Unknown"	196	265	461
"Financial Services"	131	170	301
"Energy"	129	159	288
"Basic Materials"	95	142	237
"Communication Services"	77	123	200
"Consumer Defensive"	73	96	169
"Real Estate"	15	31	46
"Utilities"	7	7	14

5 Empirical specification

The empirical strategy is residualize-then-estimate: first remove the part of returns that is explained by general market factors, then test whether what's left can be explained by the breakout-specific covariates.

In the first stage, 20-day forward returns are regressed on the cumulative four-factor returns (Mkt-RF, SMB, HML, UMD) over the same 20-day holding window, plus sector and year fixed effects. The model is fit on the 2010 to 2017 training window only. The fitted coefficients are then applied to all rows, including out-of-sample years, to construct the residualized return `fwd_ret_20d_resid`. This residual is the part of each setup's return that is not explained by market, size, value, momentum, sector, or year. It becomes the dependent variable for the second stage.

In the second stage, both OLS and quantile regression at five quantiles (τ in $\{0.10, 0.25, 0.50, 0.75, 0.90\}$) are estimated of `fwd_ret_20d_resid` on four covariates: `vol_contraction_ratio_w`, `adr_pct`, `base_duration_days`, and `rs_slope_vs_spy`. The 'w' suffix on volume contraction means the variable is winsorized at the training-window 99th percentile, which caps extreme right-tail values to limit the influence of a few outliers. The volume contraction ratio is the average volume in the second half of the consolidation window divided by the average volume in the first half, with the half-split applied exactly regardless of base length. A ratio below 1 means volume dried up as the base matured, which is the

standard Qullamaggie pattern. The headline test runs on the loose universe ($\text{mom_pct} \geq 0.80$) because it preserves a larger sample than the strict universe and gives the upper-tail quantile estimate more statistical power. The strict universe is reported in 7.1 as a sensitivity check.

6 Pre-Registered Hypothesis (locked May 3, 2026)

I pre-register a one-sided test of whether the volume contraction ratio shifts the upper-tail ($\tau=0.90$) conditional quantile of factor-residualized 20-day forward returns more than it shifts the median ($\tau=0.50$), after controlling for the 12-1 momentum factor, sector fixed effects, and year fixed effects.

The test statistic is $\beta(0.90) - \beta(0.50)$. In words: I run a quantile regression at $\tau=0.90$ to estimate how a one-unit change in volume contraction ratio shifts the 90th-percentile conditional return, then run another quantile regression at $\tau=0.50$ to estimate how the same one-unit change shifts the median conditional return, and take the difference. If volume contraction matters more for upside outliers than for typical setups, this difference should be negative (a lower volume contraction ratio, meaning volume dries up more, lifts the upper tail more than the median).

The statistic is estimated on the 2010 to 2017 training window. Inference uses a stationary block bootstrap with mean block length 30 to handle serial dependence in the panel. The hypothesis survives if (a) the 95% bootstrap CI of the statistic excludes zero with the predicted negative sign, and (b) the same negative sign holds in at least 5 of 8 expanding-window out-of-sample years (2018 to 2025).

The headline test runs on the loose universe ($\text{mom_pct} \geq 0.80$) to maximize sample size for quantile regression. The strict universe ($\text{mom_pct} \geq 0.90$ plus a gate requiring the stock to be within 15% of its 52-week high) is reported in 7.1 as a sensitivity analysis.

7 Results

On the pre-registered training window (loose universe, $n=925$, 2010 to 2017), the point estimate of $\beta(0.90) - \beta(0.50)$ on $\text{vol_contraction_ratio_w}$ is negative 0.0045. The 95% percentile bootstrap CI is [negative 0.038, positive 0.043], with a one-sided bootstrap p-value of 0.50.

The CI does not exclude zero, so the hypothesis fails on the inferential leg of the pre-registration. To check whether this failure means the true effect is zero, or whether the test is just too weak to detect a small effect, I ran a placebo bootstrap. The placebo shuffles $\text{vol_contraction_ratio_w}$ within the training window before re-estimation, which destroys any real link between volume contraction and returns by construction. The resulting null distribution is what the test statistic looks like when there is no signal at all. My actual point

estimate of negative 0.0045 sits at the 39th percentile of that null, meaning my real result is well within what pure noise produces. This indicates the test is statistically underpowered, not that the true effect is zero.

The expanding-window walk-forward, however, shows the predicted negative sign in 7 of 8 out-of-sample years from 2018 to 2025. This satisfies the second leg of the pre-registration. Since both legs are required and the inferential leg failed, the formal hypothesis does not survive.

```
In [20]: # Loose + strict primary-spec coefficient tables (OLS row + 5 QR rows).
m4 = pl.read_parquet(DATA / "m4_results.parquet")

TAUS = (0.10, 0.25, 0.50, 0.75, 0.90)
TERMS = ["const", "vol_contraction_ratio_w", "adr_pct",
         "base_duration_days", "rs_slope_vs_spy"]

def m4_panel_table(panel: str) -> pd.DataFrame:
    df = m4.filter((pl.col("panel") == panel) & (pl.col("spec") == "primary"))
    rows = []
    spec_rows = [("OLS", "OLS", None)] + [(f"tau={t:.2f}", "QR", t) for t in TAUS]
    for label, model, tau in spec_rows:
        sub = df.filter(pl.col("model") == model)
        if tau is not None:
            sub = sub.filter(pl.col("tau") == tau)
        row = {"model/tau": label}
        for term in TERMS:
            r = sub.filter(pl.col("term") == term)
            row[term] = (
                f"{float(r['coef'][0]):+.4f} ({float(r['t_or_z'][0]):+.2f})"
                if not r.is_empty() else "-"
            )
        if not sub.is_empty():
            row["n"] = f"{int(sub['n'][0]):,}"
            row["R²/pseudo-R²"] = (
                f"{float(sub['rsquared'][0]):.4f}" if model == "OLS"
                else f"{float(sub['pseudo_r2'][0]):.4f}"
            )
        rows.append(row)
    return pd.DataFrame(rows)

print("Loose panel – primary spec")
display(m4_panel_table("loose"))
print("\nStrict panel – primary spec")
display(m4_panel_table("strict"))
```

Loose panel – primary spec

	model/tau	const	vol_contraction_ratio_w	adr_pct	base_duration_days	rs_slope_vs_spy
0	OLS	+0.0742 (+3.11)	-0.0066 (-0.59)	-1.7511 (-4.38)	-0.0000 (-0.05)	-0.1997 (-0.16)
1	tau=0.10	+0.0612 (+2.12)	-0.0235 (-1.60)	-5.1747 (-10.16)	+0.0007 (+0.95)	+3.8956 (+2.80)
2	tau=0.25	+0.0623 (+2.51)	-0.0124 (-1.08)	-3.3174 (-8.13)	+0.0002 (+0.26)	-0.4099 (-0.34)
3	tau=0.50	+0.0622 (+2.80)	+0.0026 (+0.25)	-2.0743 (-5.58)	+0.0003 (+0.48)	+0.2170 (+0.18)
4	tau=0.75	+0.0638 (+2.72)	+0.0106 (+0.96)	-0.3946 (-0.97)	+0.0001 (+0.23)	-0.1827 (-0.15)
5	tau=0.90	+0.0582 (+1.66)	-0.0019 (-0.11)	+2.6284 (+4.62)	-0.0005 (-0.53)	-0.0150 (-0.01)

Strict panel – primary spec

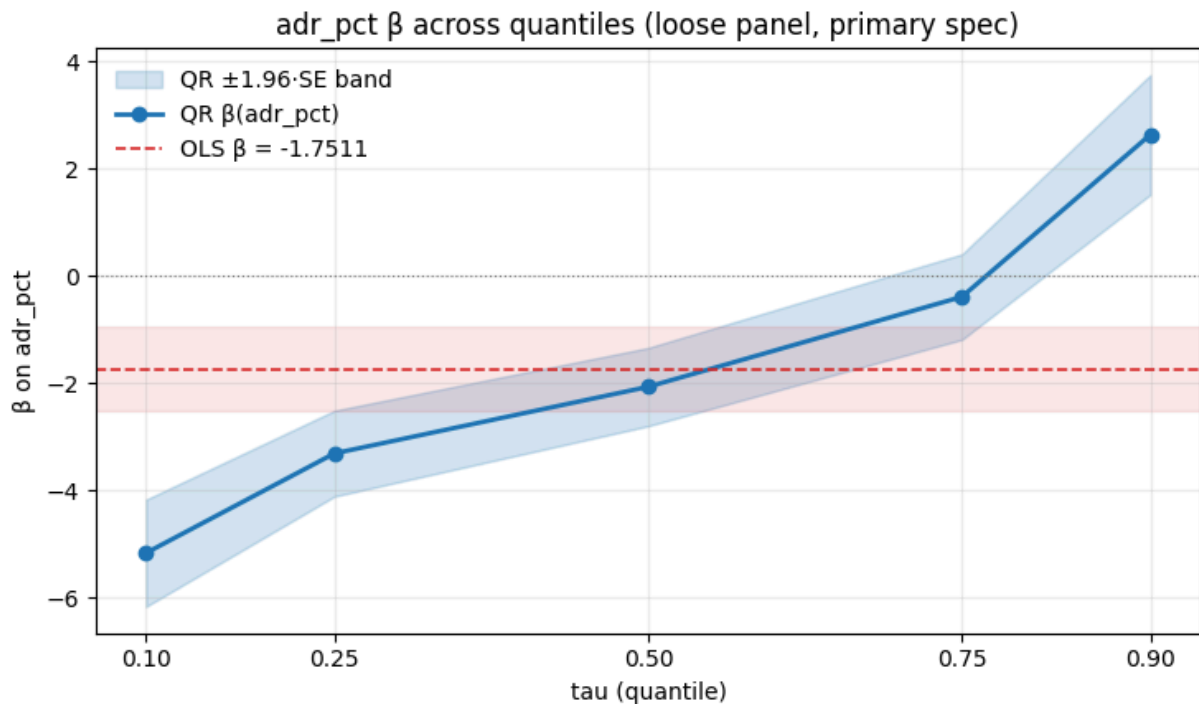
	model/tau	const	vol_contraction_ratio_w	adr_pct	base_duration_days	rs_slope_vs_spy
0	OLS	+0.0583 (+2.07)	-0.0025 (-0.19)	-1.6545 (-3.61)	+0.0003 (+0.37)	+0.0678 (+0.05)
1	tau=0.10	+0.0478 (+1.40)	-0.0172 (-0.99)	-4.8778 (-8.42)	+0.0007 (+0.81)	+1.1755 (+0.81)
2	tau=0.25	+0.0592 (+2.14)	-0.0157 (-1.21)	-3.2229 (-7.39)	+0.0002 (+0.31)	-0.2182 (-0.17)
3	tau=0.50	+0.0539 (+2.18)	-0.0005 (-0.04)	-1.9508 (-4.85)	+0.0005 (+0.82)	+0.7543 (+0.60)
4	tau=0.75	+0.0134 (+0.50)	+0.0240 (+1.91)	+0.2515 (+0.54)	+0.0006 (+0.85)	+0.6638 (+0.48)
5	tau=0.90	+0.0594 (+1.36)	-0.0018 (-0.09)	+2.4773 (+3.47)	-0.0000 (-0.04)	+1.6676 (+0.74)

```
In [21]: # Figure: adr_pct  $\beta$  across tau for the Loose panel, with  $\pm 1.96 \cdot SE$  band and OLS ref
df = m4.filter(
    (pl.col("panel") == "loose") & (pl.col("spec") == "primary")
    & (pl.col("model") == "QR") & (pl.col("term") == "adr_pct")
).sort("tau")
taus = df["tau"].to_numpy()
coefs = df["coef"].to_numpy()
ses = df["std_err"].to_numpy()

ols = m4.filter(
    (pl.col("panel") == "loose") & (pl.col("spec") == "primary")
    & (pl.col("model") == "OLS") & (pl.col("term") == "adr_pct")
)
ols_coef = float(ols["coef"][0]); ols_se = float(ols["std_err"][0])
```

```

fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.fill_between(taus, coefs - 1.96 * ses, coefs + 1.96 * ses,
                alpha=0.20, color="C0", label="QR  $\pm 1.96 \cdot SE$  band")
ax.plot(taus, coefs, marker="o", color="C0", linewidth=2, label="QR  $\beta(\text{adr\_pct})$ ")
ax.axhline(ols_coef, color="C3", linestyle="--", linewidth=1.2,
           label=f"OLS  $\beta = \{ols\_coef:+.4f\}")
ax.axhspan(ols_coef - 1.96 * ols_se, ols_coef + 1.96 * ols_se, color="C3", alpha=0.
ax.axhline(0, color="grey", linestyle=":", linewidth=0.8)
ax.set_xlabel("tau (quantile)"); ax.set_ylabel(" $\beta$  on adr_pct")
ax.set_title("adr_pct  $\beta$  across quantiles (loose panel, primary spec)")
ax.set_xticks(taus); ax.legend(loc="upper left", frameon=False); ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()$ 
```



7.1 Strict universe (robustness)

I ran the same test on the strict universe ($\text{mom_pct} \geq 0.90$, plus the 15%-of-52w-high gate). The point estimate is negative 0.0013 with a 95% bootstrap CI of [negative 0.033, positive 0.054], and walk-forward out-of-sample sign consistency drops to 4 of 8 years. Both legs of the pre-registration fail under the strict universe. The smaller sample size ($n=726$) and the additional 52-week-high filter together either reduce statistical power, reduce the strength of the effect, or both.

```

In [22]: # M5 walk-forward yearly point estimates.
m5 = pl.read_parquet(DATA / "m5_oos_results.parquet")

def m5_panel_table(panel: str):
    return (
        m5.filter(pl.col("panel") == panel).sort("year")
        .select("year", "train_end", "n_train",

```

```

        "beta_50", "beta_90", "diff", "sign_diff")
    )

print("Loose panel – expanding-window OOS")
display(m5_panel_table("loose"))
print("\nStrict panel – expanding-window OOS")
display(m5_panel_table("strict"))

```

Loose panel – expanding-window OOS

shape: (8, 7)

year	train_end	n_train	beta_50	beta_90	diff	sign_diff
i64	date	i64	f64	f64	f64	str
2018	2017-12-31	925	0.002587	-0.001947	-0.004533	"negative"
2019	2018-12-31	1074	0.000647	0.009503	0.008856	"positive"
2020	2019-12-31	1229	0.003206	-0.005099	-0.008304	"negative"
2021	2020-12-31	1556	0.010201	-0.008888	-0.019089	"negative"
2022	2021-12-31	1970	0.00569	-0.006775	-0.012465	"negative"
2023	2022-12-31	2110	0.00918	-0.005001	-0.014181	"negative"
2024	2023-12-31	2330	0.01099	-0.00654	-0.01753	"negative"
2025	2024-12-31	2692	0.012395	-0.008582	-0.020977	"negative"

Strict panel – expanding-window OOS

shape: (8, 7)

year	train_end	n_train	beta_50	beta_90	diff	sign_diff
i64	date	i64	f64	f64	f64	str
2018	2017-12-31	726	-0.000513	-0.001797	-0.001284	"negative"
2019	2018-12-31	850	-0.004078	0.017087	0.021165	"positive"
2020	2019-12-31	980	-0.001491	0.004148	0.005639	"positive"
2021	2020-12-31	1184	0.012906	0.013203	0.000297	"positive"
2022	2021-12-31	1421	0.020675	0.017604	-0.003072	"negative"
2023	2022-12-31	1527	0.014839	0.017567	0.002728	"positive"
2024	2023-12-31	1693	0.026627	0.007135	-0.019492	"negative"
2025	2024-12-31	1939	0.023225	0.005141	-0.018084	"negative"

7.2 Expanding-window exploratory

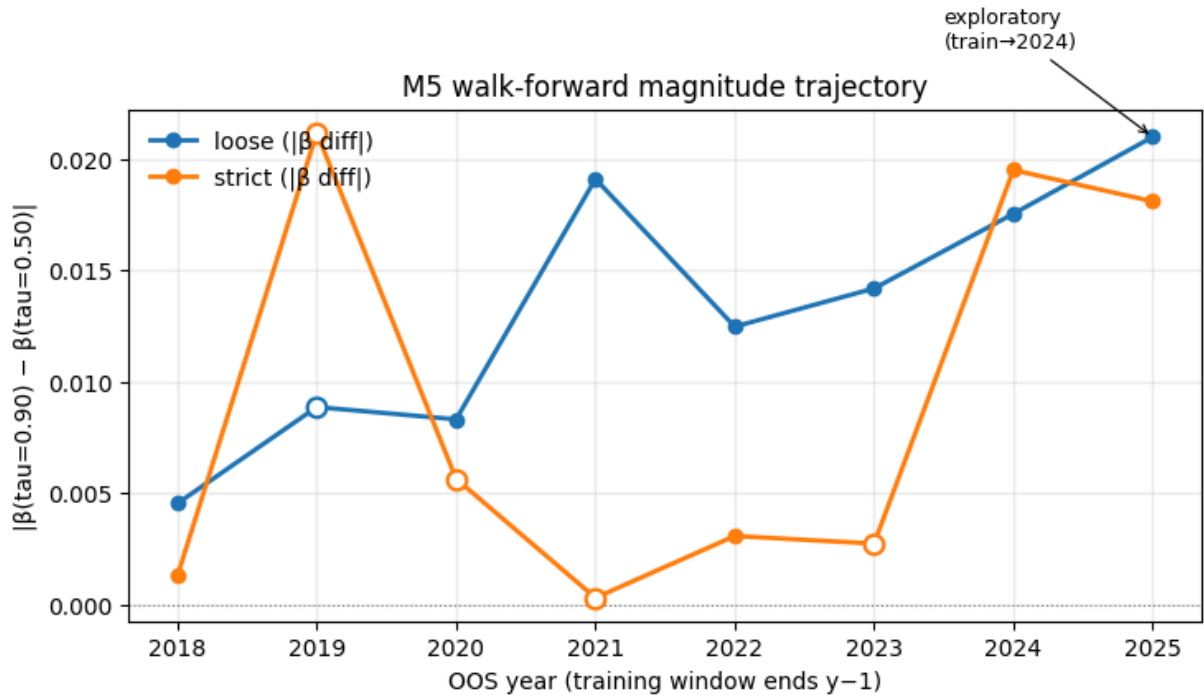
Although the pre-registered test on the 2010 to 2017 window failed, the walk-forward analysis shows a clear pattern: the magnitude of $\beta(0.90)$ minus $\beta(0.50)$ grows as the

training window expands, from negative 0.0045 at the 2017 cutoff to negative 0.0210 at the 2024 cutoff ($n = 2,692$). A separate stationary block bootstrap on the full 2010 to 2024 panel gives a 95% percentile CI of [negative 0.051, positive 0.0003], with a one-sided p-value of 0.029. The upper bound just barely overlaps zero. This is not a pre-registered test and I do not treat it as confirmatory; it is reported only to show that the failure of the headline result is consistent with a small true effect that the $n=925$ training sample lacked the power to detect, rather than with no effect at all. Resolving which of these is the case would require a longer panel or a higher signal-to-noise covariate than `vol_contraction_ratio`.

```
In [23]: # Figure:  $|\beta(0.90) - \beta(0.50)|$  trajectory across expanding windows, both panels.
fig, ax = plt.subplots(figsize=(7.5, 4.5))
for panel, color in (("loose", "C0"), ("strict", "C1")):
    sub = m5.filter(pl.col("panel") == panel).sort("year")
    years = sub["year"].to_numpy()
    diffs = sub["diff"].to_numpy()
    mags = np.abs(diffs)
    ax.plot(years, mags, marker="o", color=color, linewidth=2,
            label=f"{panel} ( $|\beta \text{ diff}|$ )")
    # Hollow markers for sign-flip years.
    for x, d, m in zip(years, diffs, mags):
        if d >= 0:
            ax.scatter([x], [m], color="white", edgecolor=color,
                       zorder=5, s=70, linewidths=1.6)

loose_25 = m5.filter((pl.col("panel") == "loose") & (pl.col("year") == 2025))
if loose_25.height:
    x = int(loose_25["year"][0]); y = abs(float(loose_25["diff"][0]))
    ax.annotate("exploratory\n(train→2024)", xy=(x, y),
               xytext=(x - 1.5, y + 0.004), fontsize=9,
               arrowprops=dict(arrowstyle="→", color="black", lw=0.8))

ax.axhline(0, color="grey", linestyle=":", linewidth=0.8)
ax.set_xlabel("OOS year (training window ends y-1)")
ax.set_ylabel(" $|\beta(\tau=0.90) - \beta(\tau=0.50)|$ ")
ax.set_title("M5 walk-forward magnitude trajectory")
ax.set_xticks(sorted(set(int(yy) for yy in m5["year"].to_list())))
ax.legend(loc="upper left", frameon=False); ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```



```
In [24]: # M6 bootstrap inference: percentile/basic CI, one-sided  $\hat{p}$ , placebo summary.
m6 = pl.read_parquet(DATA / "m6_bootstrap.parquet")

def m6_summary_row(label: str, panel: str) -> dict:
    sub = m6.filter((pl.col("label") == label) & (pl.col("panel") == panel))
    point = sub.filter(pl.col("mode") == "point")
    real = sub.filter(pl.col("mode") == "real")["diff"].to_numpy()
    plac = sub.filter(pl.col("mode") == "placebo")["diff"].to_numpy()
    theta = float(point["diff"][0])
    p25 = float(np.quantile(real, 0.025))
    p975 = float(np.quantile(real, 0.975))
    p_one = float((np.sum(real >= 0) + 1) / (real.size + 1))
    if plac.size:
        plac_mean = float(plac.mean())
        plac_std = float(plac.std(ddof=1))
        pct_null = float((plac <= theta).mean())
    else:
        plac_mean = plac_std = pct_null = float("nan")
    return {
        "panel": f"{label}/{panel}",
        "n_train": int(point["n_train"][0]),
        "θ": theta,
        "CI_lo": p25,
        "CI_hi": p975,
        "basic_lo": 2 * theta - p975,
        "basic_hi": 2 * theta - p25,
        "one_sided_p̂": p_one,
        "placebo_mean": plac_mean,
        "placebo_std": plac_std,
        "pct_null≤θ": pct_null,
    }

pd.DataFrame([
    m6_summary_row("canonical", "loose"),
```

```

m6_summary_row("canonical", "strict"),
m6_summary_row("exploratory", "loose"),
])

```

Out[24]:

	panel	n_train	θ	CI_lo	CI_hi	basic_lo	basic_hi	one_sided_p̂	place
0	canonical/loose	925	-0.0045	-0.0378	0.0425	-0.0515	0.0287	0.4955	
1	canonical/strict	726	-0.0013	-0.0329	0.0543	-0.0569	0.0303	0.5785	
2	exploratory/loose	2692	-0.0210	-0.0507	0.0003	-0.0423	0.0088	0.0293	

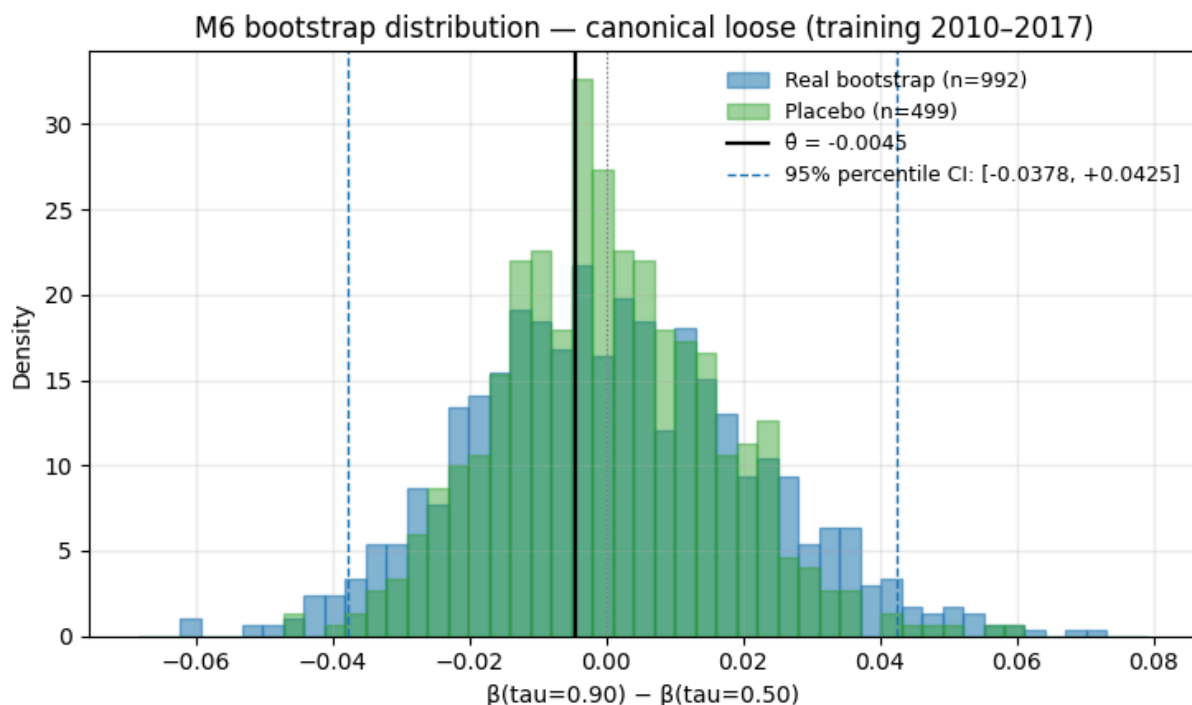
In [25]:

```

# Figure: canonical-loose bootstrap distribution overlaid with the placebo null.
sub = m6.filter((pl.col("label") == "canonical") & (pl.col("panel") == "loose"))
real_arr = sub.filter(pl.col("mode") == "real")["diff"].to_numpy()
plac_arr = sub.filter(pl.col("mode") == "placebo")["diff"].to_numpy()
theta = float(sub.filter(pl.col("mode") == "point")["diff"][0])
p25 = float(np.quantile(real_arr, 0.025))
p975 = float(np.quantile(real_arr, 0.975))

lo = float(min(real_arr.min(), plac_arr.min(), theta))
hi = float(max(real_arr.max(), plac_arr.max(), theta))
pad = 0.05 * (hi - lo) if hi > lo else 0.01
bins = np.linspace(lo - pad, hi + pad, 50)

fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.hist(real_arr, bins=bins, density=True, alpha=0.55, color="C0",
        label=f"Real bootstrap (n={real_arr.size:,})", edgecolor="C0")
ax.hist(plac_arr, bins=bins, density=True, alpha=0.45, color="C2",
        label=f"Placebo (n={plac_arr.size:,})", edgecolor="C2")
ax.axvline(theta, color="black", linewidth=1.6, label=f" $\theta = \{theta:+.4f\}")
ax.axvline(p25, color="C0", linestyle="--", linewidth=1.0,
        label=f"95% percentile CI: [{p25:+.4f}, {p975:+.4f}]")
ax.axvline(p975, color="C0", linestyle="--", linewidth=1.0)
ax.axvline(0, color="grey", linestyle=":", linewidth=0.8)
ax.set_xlabel(" $\beta(\tau=0.90) - \beta(\tau=0.50)$ ")
ax.set_ylabel("Density")
ax.set_title("M6 bootstrap distribution - canonical loose (training 2010-2017)")
ax.legend(loc="upper right", frameon=False, fontsize=9)
ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()$ 
```



7.3 adr_pct ancillary finding

A separate finding emerged from the quantile regression sweep that was not part of the pre-registered hypothesis. The covariate `adr_pct` (the average daily range over the consolidation window) shows a clean sign flip across τ on the loose panel: at $\tau = 0.10$ the coefficient is negative 5.17 with a t-statistic of negative 10.2, while at $\tau = 0.90$ the coefficient is positive 2.63 with a t-statistic of positive 4.6. The OLS coefficient is negative 1.75, which obscures this pattern entirely. This is the predominant reason quantile regression matters, which is that high daily-range setups have systematically heavier tails on both sides of the conditional return distribution, and reporting only the conditional mean would mistake a fat-tail effect for a pure downside effect.

8 Limitations

Several limitations are worth flagging.

First, the M1 setup detector achieved roughly 6.5 out of 10 on a manual chart-quality QC of a 10-row sample, after three iterations of rule refinement. A higher-quality detector would yield a cleaner conditioning event.

Second, the analysis uses daily OHLCV bars only. The reference trading framework (`breakouts.trade`) additionally gates on first-30-minute intraday volume and range; this confirmation cannot be replicated with daily data and may either over-include marginal breakouts or over-reject valid ones.

Third, relative strength is measured cross-sectionally as a within-universe momentum percentile rather than as a SPY-relative return slope. The full Qullamaggie definition uses both.

Fourth, the pre-registered training window contains only 925 loose-universe setups. The placebo result confirms that the headline test is statistically underpowered. A pre-registration that specified the full 2010 to 2024 panel as training would have had different inferential properties; whether such a window would have rejected the null is the open question 7.2 raises but does not answer.

Fifth, sector classification has 8.77% missing values (recorded as 'Unknown'), driven by yfinance fetch failures on roughly 119 of 1,773 unique tickers in the panel. These rows form their own sector bucket in the factor residualization.

9 References & data sources

Data sources:

- Daily OHLCV bars: Massive (massive.com), historical equities data, paid subscription; not redistributed in this repository.
- FF3 + UMD daily factors: Kenneth R. French data library, Dartmouth Tuck (mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html).
- SPY daily closes: yfinance.
- Ticker security type, country, and sector: yfinance.

Methodological references:

- Koenker, R. & Hallock, K. F. (2001). Quantile regression. *Journal of Economic Perspectives*
- Politis, D. N. & Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*

Trading framework reference:

- 'Qullamaggie' (Kristjan Kullamägi). Public educational materials and interviews on momentum breakout methodology.

AI disclosure:

For some components of the project, I relied on Claude Opus 4.7 via Claude Desktop and Claude Code, and it was used as a coding assistant for some of the implementation of the M0 through M6 modules. Methodological choices including the pre-registration design, hypothesis specification, residualization approach, bootstrap configuration, and analysis interpretation were entirely my own. AI was not used to directly generate any part of this writeup.

